

Programación Funcional Parcial - Tipado simple

En esta evaluación se busca modelar un chequeo de tipos sobre expresiones de un lenguaje que opera sobre números y texto. Se define un tipo `Exp` para modelar expresiones de un lenguaje de programación, un tipo `TType` para modelar los tipos de dichas expresiones, y un tipo `Value` que modela los posibles valores resultantes de reducir a las expresiones modeladas.

```
data TType = Number | Boolean | Text
```

```
type Variable = String
```

```
data Exp
```

```
  = LNum Int           -- Literal numérico
  | LText String       -- Literal de texto
  | Var Variable       -- Variable de tipo texto
  | AritOp (Int->Int->Int) Exp Exp -- Operaciones aritméticas sobre números
  | RelOp  (Int->Int->Bool) Exp Exp -- Operaciones relacionales sobre números
  | TextAritOp (String->String) Exp -- Operaciones texto a texto
  | TextNumOp (String->Int) Exp    -- Operaciones texto a número
```

```
data Value = VInt Int | VBool Bool | VText String | TypeError
```

Se define además un tipo `Cmd` para modelar un lenguaje imperativo simple que utiliza estas expresiones.

```
data Cmd
```

```
  = Skip                -- statement que no hace nada
  | StoreInput Variable String -- recuerda texto literal en la variable dada
  | PrintVar Variable   -- imprime el valor de una variable
  | Seq Cmd Cmd         -- secuencia de comandos
  | If Exp Cmd Cmd      -- alternativa entre dos comandos
```

```
data State = S (Variable->String) [String]
```

```
  --      memoria de variables stdout
```

INV_REP: La memoria de variables es una función total

```
baseEnv = const ""
```

```
envzHola = \s -> if s=="x" then "Hola" else ""
```

```
test = S baseEnv []
```

Ejercicios

1. Definir por recursión explícita las siguientes funciones

a. `typecheck :: Exp -> Maybe TType`, que dada una expresión, determina su tipo, si el mismo es correcto. Las operaciones tienen tipo correcto cuando se aplican a los argumentos del tipo que corresponde, y dan como resultado el tipo que indica qué es el resultado.

EJEMPLOS:

```
* typecheck (AritOp (+) (LNum 2) (LText "Hola")) = Nothing
* typecheck (RelOp (>) (LNum 2) (LNum 3)) = Just Boolean
```

b. `evalExp :: Exp -> (Variable->String) -> Value`, que dada una expresión y un valor para cada variable (como función total), evalúa el valor que corresponde: si la expresión no tiene tipo válido, da el valor `TypeError` y si no da el resultado correspondiente como elemento de tipo

`Value`.

EJEMPLOS:

```
* evalExp (AritOp (+) (LNum 2) (LText "Hola")) baseEnv = TypeError
* evalExp (RelOp (>) (LNum 2) (LNum 3)) baseEnv = VBool False
```

2. Dada la siguiente función:

```
tvalue :: Value -> Maybe TType
tvalue (VInt _) = Just Number
tvalue (VBool _) = Just Boolean
tvalue (VText _) = Just Text
tvalue TypeError = Nothing
```

demostrar la siguiente propiedad para las definiciones del ejercicio 1:
para todo env, `typecheck = tvalue . flip evalExp env`

AYUDA: desarrollar solamente los casos `Var` y `RelOp`.

4. Dar los tipos y escribir los esquemas de recursión estructural y primitiva para `Exp`.

5. Escribir las siguientes funciones utilizando esquemas, y sin utilizar recursión explícita en ninguna función diferente de los esquemas. Pueden utilizarse las funciones del ejercicio 4.

a. `simplifyCmd :: Cmd -> Cmd`, que describe un comando equivalente al dado, pero que no utilice `Skip` en ninguna secuencia, y que elimine aquellas alternativas cuyas condiciones tienen comparaciones entre 2 literales.

EJEMPLOS:

```
* simplifyCmd (Seq Skip (Seq (PrintVar "x") Skip)) = PrintVar "x"
* simplifyCmd (If (RelOp (>) (LNum 2) (LNum 3))
  (PrintVar "x")
  (PrintVar "y"))
  = (PrintVar "y")
```

b. (BONUS) `evalCmd :: Cmd -> State -> Maybe State`, que ejecuta el comando dado. En la ejecución, los comandos de asignación modifican la memoria, los comandos de impresión modifican la salida, y los comandos de secuencia y alternativa se comportan como los usuales en un lenguaje de programación secuencial.

EJEMPLOS:

```
* evalCmd (PrintVar "x") baseState = Just (S baseEnv [])
* evalCmd (Seq (StoreInput "x" "Hola")
  (PrintVar "x")) baseState = Just (S envXHola ["Hola"])
* evalCmd (PrintVar "x") (S envXHola []) = Just (S envXHola ["Hola"])
* evalCmd (If (AritOp (+) (LNum 2) (LText "Hola"))
  (PrintVar "x")
  (PrintVar "y")) baseEnv = Nothing
```